# Tensor Network Simulation of the Heisenberg Model on Heavy-Hex Lattices using AMD GPUs

Paolo D.
AMD Research

July 10, 2026

**Abstract**

We extend CppSim, a high-performance C++/HIP tensor network simulator, to the heavy-hex lattice geometry used in IBM superconducting quantum processors. The heavy-hex graph (degree-2 and degree-3 sites, 3-colorable gate schedule) is implemented as a drop-in grid class with no changes to the gate application or belief propagation kernels. We simulate the isotropic Heisenberg model $H = J\sum_{\langle ij\rangle}(\mathbf{X}_i\mathbf{X}_j + \mathbf{Y}_i\mathbf{Y}_j + \mathbf{Z}_i\mathbf{Z}_j)$ in the Heisenberg picture using Pauli transfer matrices, and study the autocorrelation function $C(t) = \langle Z_c(t)Z_c(0)\rangle$ and operator lightcone on the `heavy_hex_3x3` graph (68 sites, 76 bonds). We demonstrate chi convergence up to $\chi =$ **[TODO: X]** on an AMD MI100 GPU (32 GB), and show a **[TODO: X]**× performance advantage of the C++/HIP implementation over a Julia/AMDGPU.jl reference at saturated bond dimension.

## 1 Introduction

Heavy-hex is the lattice topology used in IBM's superconducting quantum processors [IBM heavy-hex]. Each qubit site has degree 2 or 3, and the graph admits a natural 3-coloring of its bonds, enabling parallel gate application within each color class.

Classical tensor network (TN) simulation of quantum circuits on heavy-hex graphs serves two purposes: (i) validation of quantum computations for small system sizes where exact simulation is feasible, and (ii) computation of noise-free reference values for quantum error mitigation protocols such as the Multi-Product Formula (MPF) [MPF ref].

In this paper we show that CppSim [cppsim], originally developed for rectangular 2D Ising dynamics, extends naturally to heavy-hex topology with minimal code changes. The hot-path kernels — gate application, belief propagation, workspace management — are topology-agnostic and require no modification. Only the grid construction and Hamiltonian gate matrix need to be defined for the new geometry.

## 2 Heavy-Hex Lattice and Hamiltonian

### 2.1 Graph structure

The `heavy_hex_`$n_x \times n_y$ graph has:

- $(n_x + 1)(n_y + 1)$ *junction* sites (degree 3)
- remaining *arm* sites (degree 2) connecting adjacent junctions
- $2n_xn_y + n_x + n_y$ bonds total

For $n_x = n_y = 3$: 16 junction sites, 52 arm sites, 76 bonds. The graph is bipartite and 3-edge-colorable — bonds partition into 3 color classes of sizes 25, 25, 26, corresponding to the natural gate layers of the IBM hardware schedule.

Site tensors have shapes $[\chi^2, D]$ (arm) and $[\chi^3, D]$ (junction), where $D = 4$ is the Pauli basis dimension $\{I, X, Y, Z\}$. The memory cost is dominated by junction sites: $16 \times \chi^3 \times D \times 4\,\mathrm{B}$ (float32).

### 2.2 Implementation

The `FlatHeavyHexGrid` class inherits from `Grid` and populates the same `Site/Bond/Gate` data structures used by the rectangular grid. The color class assignment follows the hardware gate schedule (JSON layers 0, 1, 2). No changes are required in `gate.hpp`, `measures.hpp`, or `workspace.hpp` beyond parametrizing the workspace by `max_degree=3` instead of 4.

### 2.3 Hamiltonian and gate matrix

We simulate the isotropic Heisenberg (XXX) model:

$$H = J\sum_{\langle ij\rangle}\left(X_iX_j + Y_iY_j + Z_iZ_j\right) \tag{1}$$

with uniform coupling $J = 1$ and time step $\delta t = 0.1$.

In the Heisenberg picture with Pauli basis, the two-site gate $U = e^{-i\theta(X\otimes X + Y\otimes Y + Z\otimes Z)}$ ($\theta = J\delta t$) is represented

by a $16 \times 16$ real Pauli transfer matrix (PTM) with block structure:

$$\{II, XX, YY, ZZ\} \to \text{invariant (diagonal} = 1) \quad (2)$$
$$\{IX, XI, YZ, ZY\} \to 4 \times 4 \text{ mixing block} \quad (3)$$
$$\{IY, YI, XZ, ZX\} \to 4 \times 4 \text{ mixing block} \quad (4)$$
$$\{IZ, ZI, XY, YX\} \to 4 \times 4 \text{ mixing block} \quad (5)$$

with elements $\alpha = \cos^2(2\theta)$, $\beta = \sin^2(2\theta)$, $\gamma = \sin(2\theta)\cos(2\theta)$. This was verified against a Julia reference implementation.

# 3 Autocorrelation and Chi Convergence

## 3.1 Autocorrelation function

We initialize the Heisenberg-picture operator as $Z$ at a single center site (site 13, a degree-3 junction at the graph midpoint) and $I$ elsewhere. The autocorrelation $C(t) = \langle Z_c(t) Z_c(0) \rangle$ is computed via the Bethe approximation after each Trotter step.

## 3.2 Chi convergence

**[TODO: Insert ct_convergence.png figure]**

**[TODO: Insert convergence table from plot_ct_convergence.py]**

Figure **??** shows $C(t)$ for $\chi = 50, 100, 150, 200, 250, 300, 350, 400, 430$ over 30 Trotter steps. The result converges to **[TODO: X decimal places]** at $\chi =$ **[TODO: X]**, indicating that $\chi_{\max} =$ **[TODO: X]** is sufficient for faithful simulation on this graph.

The non-monotonic behaviour of $C(t)$ — a decrease followed by partial recovery — is a genuine quantum recurrence of the XXX model on the degree-3 lattice, confirmed by its independence from $\chi$.

## 3.3 Operator lightcone

**[TODO: Insert lightcone_sites_spatial_site13.png figure (selected cycles)]**

Figure **??** shows the non-identity Pauli weight $n(v, t)$ at each site for selected cycles. The operator spreads radially outward from the center junction, following the 3-fold symmetry of the hexagonal lattice. The wavefront reaches the boundary sites by cycle **[TODO: X]**, defining the computational window within which the simulation is exact regardless of $\chi$ truncation at the boundary.

# 4 Performance

## 4.1 Chi scaling

**[TODO: Insert timing_vs_chi.png and chi_growth.png]**

Table **??** shows the wall-clock time per Trotter cycle at saturated bond dimension for each $\chi$ value. The dominant cost is the Householder QR decomposition ($\mathcal{O}(mn^2)$ where $m = \chi^2 D$ for degree-3 sites), which exceeds the SVD cost above $\chi \approx$ **[TODO: X]** cycles.

**[TODO: Insert chi scaling table]**

## 4.2 CppSim vs Julia/AMDGPU.jl

**[TODO: Insert comparison table (10x speedup result)]**

At saturated bond dimension ($\chi = 430$, cycles 8–10), CppSim achieves a $\sim$10$\times$ speedup over the Julia/AMDGPU.jl reference implementation on the same AMD MI100 GPU (32 GB HBM2). Both use Householder QR and float32 arithmetic. The speedup arises from: (i) zero GPU memory allocation during simulation (pre-allocated workspace), (ii) direct HIP/rocBLAS calls without framework dispatch overhead, and (iii) on-device `pseudo_sqrt` kernel eliminating CPU–GPU transfers.

# 5 Conclusion

We have demonstrated that CppSim, a C++/HIP tensor network simulator, extends to the heavy-hex lattice with minimal code changes. The topology-agnostic gate and belief propagation kernels accept any `Grid`-derived class, and the heavy-hex grid requires only a new site/bond/color construction.

On an AMD MI100 (GPU-B, 32 GB), we achieve **[TODO: exact]** simulation up to $\chi =$ **[TODO: X]** on the $3 \times 3$ heavy-hex graph (68 sites), and demonstrate $\sim$10$\times$ speedup over a Julia reference implementation at saturated bond dimension.

The operator lightcone visualization — computed as a byproduct of the simulation with no additional cost — provides an intuitive picture of operator spreading on the hexagonal lattice and defines the computational window for quantum error mitigation protocols.

Future work will extend to larger graphs ($2 \times 2$, $4 \times 4$) on higher-memory hardware (MI355X, 309 GB), and implement interleaved belief propagation to match Julia's online BP approach and eliminate BP trap effects.


# Acknowledgments

**[TODO: Acknowledgments]**